

SDEV 2351

DAY 7: ROUTE GUARDS AND NAVIGATION WITH STATE

CONTROLLING PAGE ACCESS WITH FLUXOR STATE

Session Objectives

1. Implement a route guard that checks Fluxor store state before allowing page access
2. Use `OnInitialized` (or `OnInitializedAsync` for async service calls) and `NavigationManager` to redirect users based on state conditions
3. Choose between UI hiding and route guards for access control scenarios

Today's Schedule

PHASE	TIME	WHAT'S HAPPENING
Connecting	10 min	Empty cart scenario + prior knowledge recall
Exploring	30 min	Guard pattern + cart checkout guard demo
Practicing	50 min	Cart guard + role-based guard + testable extraction
Assessing	10 min	Exit ticket + Day 8 preview

PHASE 1 · 10 MINUTES

CONNECTING

From state management to access control

The Checkout Problem

A user clicks "**Checkout**" but their cart is empty. What should happen?

OPTION A

Show the checkout page with an "empty cart" banner. The page renders — the user sees content they can't use.

OPTION B

Redirect to the shop page before the checkout page ever renders. The user never sees an empty checkout.

Enterprise apps typically redirect. Option B prevents the user from seeing a page they can't use. That's a **route guard**.

Building on What You Know

- From **Days 3-6**: You built centralized state with Fluxor. Cart state belongs in the store because it survives navigation and is shared across components.
- From **Days 1-2**: You learned `NavigationManager.NavigateTo()` for programmatic navigation.
- **Today**: You combine those two tools to control which pages a user can access.

PHASE 2 · 30 MINUTES

EXPLORING

The route guard pattern

What Is a Route Guard?

A **route guard** is logic that runs when a page loads and redirects the user if a condition is not met.

In Blazor, guards live in `OnInitialized` — it runs **before the first render**.

Three ingredients:

1. Inject `IState<T>` to read the condition
2. Inject `NavigationManager` for redirecting
3. Check the condition in `OnInitialized` — redirect if it fails

The Guard Pattern

```
@inject IState<CartState> CartState
@inject NavigationManager Navigation

@code {
    protected override void OnInitialized()
    {
        if (CartState.Value.Items.Count == 0)
        {
            Navigation.NavigateTo("/shop");
            return; // Critical – stop executing
        }
    }
}
```

Full code: [see reference-sheet.md](#) or your IDE

Guard vs. UI Hiding

APPROACH	WHEN TO USE	RENDERS PAGE?
Route Guard (<code>onInitialized</code> + <code>redirect</code>)	Page-level: user should not see this page at all	No
UI Hiding (<code>@if</code> in markup)	Element-level: hide a button or section	Yes

Examples:

- Empty cart to checkout — **guard** (page-level, redirect)
- Hide "Delete" button for non-admins — **UI hiding** (element, same page)
- Admin dashboard for non-admin users — **guard** (page-level, redirect)

Refactoring for Testability

Extract the decision logic into a static class so it can be unit tested without rendering a component:

```
public static class CartGuard
{
    public static bool ShouldRedirect(CartState state)
    {
        return state.Items.Count == 0;
    }
}
```

The component calls `CartGuard.ShouldRedirect(CartState.Value)` — same behavior, but now a test can verify the logic independently.

Demonstration: Cart Checkout Guard

Watch your instructor build the cart checkout guard live:

- The **problem** — `/checkout` renders even with an empty cart
- **Add the guard** — inject `NavigationManager`, override `OnInitialized`, redirect to `/shop`
- **Extract `CartGuard`** — move the decision into a testable static class

The finished cart guard ships **pre-wired in your practice starter** as a worked reference — open `Checkout.razor` to study it.

PHASE 3 · 50 MINUTES

PRACTICING

Hands-on guard implementation

Warm-Up: BYSResult-Fluxor Review (10 min)

This slot does **two things for two groups** — your instructor will split the time:

- **Review (whole class, ~4 min):** a quick recap of the `Result<T>` pattern from Day 6 — a service returns `Result<T>` instead of throwing; the caller checks `.IsSuccess` for `.Value` or `.IsFailure` for `.Errors`.
- **Catch-up (~6 min):** if you have unfinished Day 6 Practice 2 work, finish it now. If you finished Day 6, re-read the pre-wired checkout guard in the starter instead.

How We Practice Today

The route guard is a **new pattern**, so we ramp up in three rungs:

- **I-do** — you watched me build the cart guard in the Demonstration.
- **We-do** — we build the first part of the admin guard **together**, at my pace.
- **You-do** — you finish Practice 1 and do Practice 2 on your own.

No new code or content in the we-do step — just the same guard pattern, built together first.

Practice 1: Admin Dashboard Guard (25 min)

GUIDED OPENING → INDEPENDENT

Starter provides: `UserState` with `Role` property ("User" / "Admin"), `Login.razor` that dispatches `SetUserRoleAction`, `AdminDashboard.razor` with content but no guard

Guided opening — we-do (10 min): together we add `@inject IState<UserState>` + `@inject NavigationManager`, override `OnInitialized`, and write the first guard check — `if (UserState.Value.Role ≠ "Admin")`.

Independent — you-do (15 min): from there, on your own —

- Redirect non-admin users to `/access-denied` and `return`
- Create `AccessDenied.razor` with a message and a link back to Home

Test: Visit `/admin` as a non-admin — you should be redirected to `/access-denied`.

Practice 2: Form Wizard Guards (10 min)

INDEPENDENT CHALLENGE

Guard a three-step form wizard so users cannot skip ahead:

- **FormStep2.razor** — redirect to `/form/step1` if step 1 is not complete
- **FormStep3.razor** — redirect to the earliest incomplete step if steps 1 and 2 are not both complete
- Each guard checks `FormState.Value.CompletedSteps`

The same three-ingredient guard pattern, chained across multiple pages.

Troubleshooting

PROBLEM	CAUSE	FIX
Guard runs but page still flashes	Using <code>OnAfterRender</code> instead of <code>OnInitialized</code>	Move guard logic to <code>OnInitialized</code>
Infinite redirect loop	Guard redirects to a page that also has a guard	Ensure the target page has no conflicting guard
Code runs after redirect	Missing <code>return</code> after <code>NavigateTo</code>	Always <code>return;</code> after calling <code>NavigateTo</code>

PHASE 4 · 10 MINUTES

ASSESSING

Summary

Today you learned to:

1. **Implement a route guard** that checks Fluxor store state before allowing page access
2. Use `OnInitialized` and `NavigationManager` to redirect users based on state conditions
3. **Choose between UI hiding and route guards** based on whether access control is page-level or element-level

Exit Ticket

In one sentence:

Why does guard logic go in `OnInitialized` instead of `OnAfterRender`?

What's Next

Day 8: State Persistence Patterns

You guarded the checkout page today. The guard checks if the cart has items. But what happens when the user **refreshes the browser**?

The Fluxor store resets — all state is lost. The cart is empty. The guard blocks them.

Tomorrow we fix this with **state persistence** — saving state to the browser's `localStorage` so it survives refreshes.